

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3067 – Redes de Computadoras

Sección 10

Ing. Kevin Velásquez



Proyecto 1 - MCP Chatbot

Entrega Final

Paula Daniela de León Godoy - 23202

GUATEMALA, 19 de septiembre de 2026

Especificación del servidor MCP desarrollado

Para el proyecto se implementó un servidor MCP local hecho para utilizar en la industria de farmacia. Se trata de un chatbot que, con base en los síntomas reportados por un cliente, recomienda medicamentos del catálogo y permite realizar la compra. El servidor está escrito en Python y utiliza JSON-RPC 2.0 de forma manual sobre stdio, sin utilizar SDKs de MCP como FastMCP.

Transporte y formato de mensajes

El transporte utilizado es stdio. Cada mensaje JSON-RPC es un objeto JSON completo en una sola línea de texto, delimitado por salto de línea, sin saltos de línea embebidos dentro del mensaje. El servidor lee de la entrada estándar línea por línea, y escribe cada respuesta como una línea en la salida estándar.

Método initialize

Es el primer intercambio del protocolo. El cliente envía sus capacidades y la versión de protocolo que soporta, y el servidor responde con las suyas, incluyendo que ofrece la capacidad tools.

```
{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2025-06-18","capabilities":{"tools":{}},"clientInfo":{"name":"..."},"version":"..."}}
```

```
{"jsonrpc":"2.0","id":1,"result":{"protocolVersion":"2025-06-18","capabilities":{"tools":{}},"serverInfo":{"name":"pharmacy-mcp"},"version":"1.0.0"}}
```

Notificación notifications/initialized

Es enviada por el cliente después de recibir la respuesta de initialize. No lleva id y el servidor no responde a este mensaje, únicamente lo descarta, que es el comportamiento estándar para notificaciones en JSON-RPC 2.0.

Método tools/list y parámetros de cada tool

Devuelve las tools disponibles junto con su esquema de entrada en formato JSON Schema, que es lo que permite al LLM anfitrión saber qué argumentos debe enviar en cada llamada.

Tool	Parámetros	Descripción
search_by_symptom	symptom - string, requerido	Recomienda medicamentos según un síntoma
get_medication_details	name - string, requerido	Detalle completo de un medicamento
purchase_medication	name - string, requerido quantity - integer >= 1, requerido	Compra un medicamento y descuenta stock

Método tools/call

Ejecuta una tool específica pasando su nombre y argumentos. La respuesta incluye un arreglo content con el resultado en texto y una bandera isError.

```
{"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "search_by_symptom", "arguments": {"symptom": "fiebre"}}
```

```
{"jsonrpc": "2.0", "id": 3, "result": {"content": [{"type": "text", "text": "..."}], "isError": false}}
```

Manejo de errores de negocio y errores de protocolo

Hasta este punto, se distinguen dos tipos de fallo, los errores de negocio, como un medicamento inexistente, sin stock suficiente o que requiere receta, no rompen el protocolo JSON-RPC. La llamada se resuelve exitosamente y el resultado trae `isError` en `true` junto con el texto que explica el motivo, porque el LLM necesita leer ese texto para poder explicárselo al usuario final. Los errores de protocolo, como un método inexistente, JSON inválido o una tool desconocida, sí usan el campo `error` de JSON-RPC, con los códigos estándar de la especificación.

Código	Significado
-32700	Parse error, JSON inválido
-32601	Method not found
-32602	Invalid params, tool desconocida
-32603	Internal error

Reglas validadas en `purchase_medication`

- El medicamento debe existir en el catálogo.
- La cantidad debe ser un entero mayor a 0. Se excluye explícitamente el tipo `bool`, ya que en Python es subclase de `int` y de lo contrario pasaría como cantidad válida.
- El medicamento no debe requerir receta médica.
- Debe existir stock suficiente para la cantidad solicitada.

Además, se incluye en el proyecto el archivo de documentación `SPEC.md` con la especificación completa y ejemplos adicionales, y `test_client.py` como cliente de prueba independiente que ejerce el protocolo completo.

Servidor MCP remoto, chatbot anfitrión y análisis de Wireshark

Servidor MCP remoto

El mismo servidor de farmacia se desplegó de forma remota, usando exactamente la misma lógica de protocolo y de negocio definida en `protocol.py` y `catalog.py`. Para lograr esto sin duplicar código, primero se separó la lógica de MCP del transporte, dejando `protocol.py` sin ninguna dependencia de `stdio`, y luego se agregó un segundo punto de entrada que expone esa misma lógica sobre HTTP, usando la librería estándar de Python. El despliegue se hizo en un contenedor Docker sobre Render.

La diferencia entre ambos transportes es únicamente el medio de entrada y salida de los mensajes. En el servidor local, cada mensaje JSON-RPC es una línea de texto sobre `stdin` y `stdout`. En el servidor remoto, cada mensaje viaja como el body de una petición HTTP POST, con encabezado `Content-Type application/json`, y una notificación sin id responde con el código HTTP 204 en vez de simplemente no escribir nada.

POST / HTTP/1.1

Host: proyecto1-redes-chatbot.onrender.com

Content-Type: application/json

```
{"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {"protocolVersion": "2025-06-18", "capabilities": {}, "clientInfo": {"name": "...", "version": "..."}}}
```

Chatbot anfitrión

El chatbot se construyó como una aplicación de consola en Python que conecta con la API de Anthropic y coordina múltiples clientes MCP al mismo tiempo. Mantiene el contexto de la conversación mandando el historial completo de mensajes en cada llamada a la API, ya que la API no conserva memoria entre peticiones por su cuenta. Cada interacción con un servidor MCP, tanto la solicitud como la respuesta, queda registrada en un archivo de log en formato JSON Lines, cumpliendo con mantener y mostrar un registro de todas las interacciones.

El chatbot integra cuatro servidores MCP de forma simultánea, el servidor local de farmacia, el servidor remoto desplegado en Render, y los servidores oficiales de Filesystem y Git de Anthropic. Para lograr esto sin repetir código por cada servidor, se construyó un cliente MCP genérico sobre `stdio` y otro equivalente sobre HTTP, ambos con la misma interfaz pública, de forma

que el resto del chatbot no necesita distinguir con qué tipo de servidor está hablando en un momento dado.

Para ello, esta el escenario solicitado de crear un repositorio, crear un archivo README, agregarlo al staging y realizar un commit, todo a partir de una sola instrucción en lenguaje natural dada al chatbot, resuelta mediante llamadas encadenadas a las tools de Filesystem y Git sin intervención manual adicional.

Análisis de la comunicación con Wireshark

Se capturó el tráfico entre el chatbot y el servidor MCP remoto usando Wireshark, exportando las llaves de sesión TLS mediante la variable SSLKEYLOGFILE, lo cual permitió descifrar el contenido de aplicación de una conexión HTTPS generada por el propio equipo donde se ejecutó la captura.

Clasificación de mensajes JSON-RPC

Frame	Tipo JSON-RPC	Método	Dirección
327	Request	initialize	Cliente hacia servidor
363	Response	initialize	Servidor hacia cliente
366	Notification, sincronización	notifications/initialized	Cliente hacia servidor
368	Confirmación HTTP, sin contenido JSON-RPC	no aplica	Servidor hacia cliente
370	Request	tools/list	Cliente hacia servidor
375	Response	tools/list	Servidor hacia cliente
378	Request	tools/call	Cliente hacia servidor
381	Response	tools/call	Servidor hacia cliente
384	Request	tools/call	Cliente hacia servidor
389	Response	tools/call	Servidor hacia cliente

El criterio de clasificación se basa directamente en la especificación JSON-RPC 2.0. Todo mensaje con el campo id presente se clasifica como Request si espera una respuesta, y su correspondiente Response se identifica porque contiene result o error con el mismo valor de id. El único mensaje sin campo id observado es notifications/initialized, clasificado como mensaje de

sincronización, ya que no espera ni recibe respuesta a nivel de protocolo. El frame 368, se obtiene una respuesta a nivel de transporte HTTP, con código 204, aunque a nivel de JSON-RPC la notificación nunca esperó contenido de vuelta, lo cual se pone la diferencia entre completar un ciclo de petición y respuesta a nivel de transporte, y realmente esperar una respuesta con contenido a nivel de la aplicación.

Análisis por capas del modelo OSI y TCP/IP

En la capa de enlace, cada trama capturada corresponde a Ethernet, con la dirección MAC destino apuntando al gateway local y no al servidor remoto, dado que ambos extremos se encuentran en redes distintas.

En la capa de red, el protocolo es IP. La dirección IP origen, 172.20.10.2, corresponde al equipo desde donde se generó el tráfico, y la dirección destino, 216.24.57.18, corresponde a la red de borde de Render que resuelve el dominio del servidor remoto.

En la capa de transporte, se observa el establecimiento de una conexión TCP mediante el intercambio de tres pasos, hacia el puerto 443. Sobre esa misma conexión se establece después el protocolo TLS, con su propio intercambio de mensajes de negociación y llaves, antes de que pueda viajar cualquier dato de la capa de aplicación. El archivo de llaves de sesión exportado permitió a Wireshark reconstruir ese contenido cifrado.

En la capa de aplicación conviven dos protocolos superpuestos. HTTP transporta cada mensaje como una petición POST con su respectiva respuesta, y dentro del cuerpo de cada una viaja JSON-RPC, que es el protocolo de aplicación propio de MCP. Esto da que, MCP se apoya en JSON-RPC como formato de mensaje, y este a su vez puede transportarse sobre HTTP en el caso remoto o directamente sobre stdio en el caso local, sin que la lógica de protocolo ni de negocio deba cambiar entre ambos casos.

Conclusiones y comentario sobre el proyecto

Construir el chatbot anfitrión mostró que la parte más delicada de coordinar múltiples servidores MCP no es hablar el protocolo con cada uno por separado, sino decidir cómo combinar sus catálogos de herramientas sin que el LLM confunda cuál servidor debe atender cada llamada. Registrar cada tool bajo su nombre en un mismo mapa de despacho resultó suficiente para el alcance de este proyecto, aunque queda como limitación conocida el caso en que dos servidores expongan una tool con el mismo nombre, situación que aquí se hizo evidente al tener el mismo servidor de farmacia corriendo tanto local como remoto al mismo tiempo.

Separar la lógica de protocolo del transporte antes de agregar el servidor remoto resultó ser la decisión que más tiempo ahorró en esta segunda etapa. Al llegar el momento de exponer el servidor por HTTP, no fue necesario tocar ni una línea de las reglas de negocio que se tenían

propuestas previamente, únicamente se agregó un segundo punto de entrada que reutiliza la misma función central de despacho de mensajes.

También en el proceso, se hicieron cambios a lo planeado. Se intentó primero con Google Cloud Run y después con Cloudflare Containers, y ambos, aunque son más utilizados, su uso no funcionó con el proyecto, entonces se utilizó Render, que se utilizó sin ningún problema. Esto fue también gran parte en el uso de Docker, que daba un soporte que en el proceso el cambiar de proveedor de nube fue un problema solo de configuración, no de código.

El análisis con Wireshark ayudo también en la visualización practica del codigo. Sin exportar las llaves de sesión TLS, la captura solo habría mostrado tráfico cifrado sin ningún valor para el análisis pedido, y fue necesario investigar el mecanismo de SSLKEYLOGFILE para tener descrita los resultados sin la necesidad de ponerlo por memoria lo que dio de resultado.

En conjunto, el proyecto exigió de una forma extensa utilizando el funcionamiento de la terminal, la integración del chatbot y su razonamiento de el LLM en medio y sus resultados obtenidos en la nube, sin cambios drásticos en la lógica interna del programa.